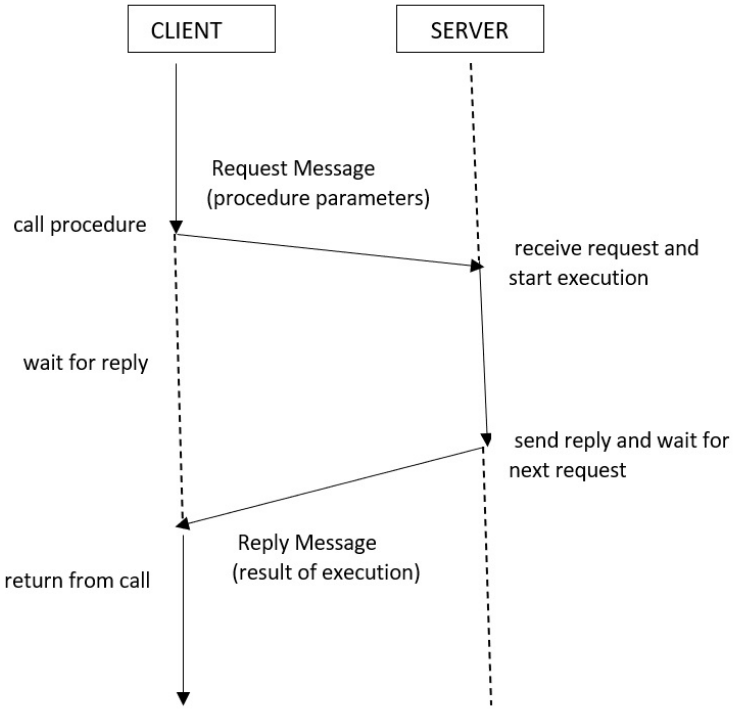
Module 2

Q. What is RPC? Explain working of RPC in detail.? Explain how transparency is achieved in RPC.

=>



Remote Procedure Call

1. Remote Procedure Call (RPC) is a method used in distributed systems to call a function that is running on another computer.
2. RPC allows a programmer to use a remote function in the same way as a local function.
3. RPC hides the network communication details from the user and programmer.
4. RPC follows the client-server model where the client requests a service and the server provides the result.
5. RPC makes distributed programming easier because developers do not need to write complex networking code.
6. RPC is commonly used in web services, cloud computing, and distributed applications.
7. An example of RPC is when a banking app requests account details from a remote server.

Components of RPC

1. The client is the program that requests a service.
2. The server is the program that provides the service.
3. The client stub acts as a helper on the client side.
4. The server stub acts as a helper on the server side.
5. The RPC runtime system manages communication between client and server.

6. The network connects the client and server computers.

Working of RPC (Step by Step)

1. The client program calls a function that looks like a normal local function call.
2. The client stub receives the request and converts the data into a message format, which is called marshalling.
3. The client stub sends the request message to the server through the network.
4. The server runtime system receives the request and passes it to the server stub.
5. The server stub converts the message back into original data, which is called unmarshalling.
6. The server stub calls the actual function on the server machine.
7. The server function performs the required task and produces the result.
8. The result is sent back to the server stub.
9. The server stub converts the result into a message and sends it to the client.
10. The client stub receives the message and converts it back into the result.
11. The client program gets the result as if the function was executed locally.

Example: A weather app requests temperature data from a remote weather server using RPC.

How Transparency is Achieved in RPC

Transparency means hiding the complexity of the distributed system from users.

1. Access Transparency

1. RPC allows remote functions to be called like local functions.
2. The programmer does not need to worry about network communication.
3. The stub hides the internal communication process.
4. Example: Calling a remote login function like a normal function.

2. Location Transparency

1. RPC hides the location of the server from the client.
2. The client does not need to know where the server is located physically.
3. The system automatically finds the server using binding mechanisms.
4. Example: Using cloud storage without knowing the server location.

3. Communication Transparency

1. RPC hides message passing and data transfer details.
2. The runtime system automatically manages communication between machines.
3. Programmers do not need to use low-level network programming.
4. Example: Sending data without writing socket code.

4. Failure Transparency (Limited)

1. RPC may handle some failures using retry and timeout mechanisms.
2. The system tries to recover automatically from communication errors.
3. However, complete failure hiding is not always possible.
4. Example: Automatic retry when a request fails due to network delay.

Q. Differentiate between RPC and RMI

=>

Feature	RPC (Remote Procedure Call)	RMI (Remote Method Invocation)
Definition	RPC is a communication technique that allows a program to call a procedure on a remote computer as if it is local.	RMI is a Java-based technology that allows an object to call methods of an object located on another machine.
Programming Language	RPC is language independent and can be implemented using different programming languages.	RMI is mainly used in Java and supports only Java programs.
Communication Type	RPC follows procedural programming concepts.	RMI follows object-oriented programming concepts.
Data Transfer	RPC transfers simple data types and structures between client and server.	RMI transfers objects as parameters and return values.
Complexity	RPC is simpler compared to RMI.	RMI is more complex because it supports object-oriented features.
Platform Dependency	RPC can work across different platforms and systems.	RMI works mainly within Java environments.
Security	RPC provides basic security mechanisms.	RMI provides better security using Java security features.
Stub Usage	RPC uses client stub and server stub for communication.	RMI uses stub and skeleton (older versions) or dynamic proxies.
Performance	RPC is generally faster because it handles simple procedure calls.	RMI may be slower due to object serialization overhead.
Object Support	RPC does not support full object-oriented features.	RMI supports object-oriented features such as inheritance and polymorphism.
Examples	Examples include Sun RPC and gRPC.	Example includes Java RMI used in distributed Java applications.

Q. Explain Group Communication and 1-M and M-1 Group Communication

=>

Group Communication in Distributed Systems

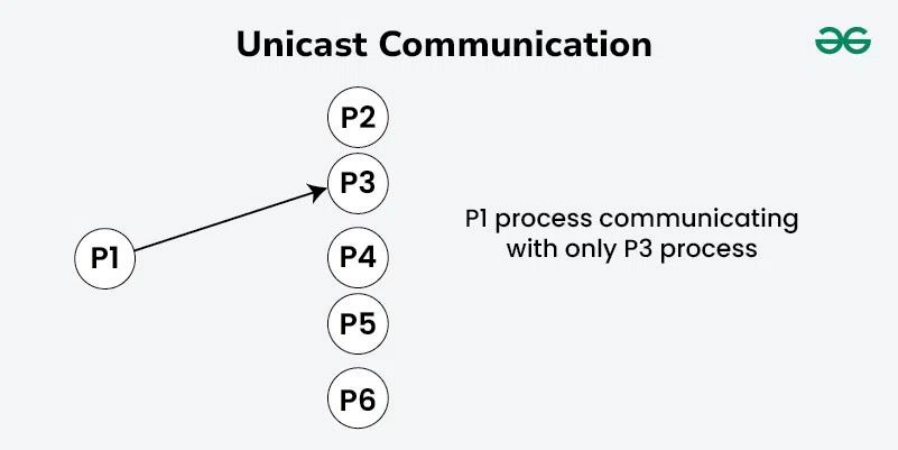
1. Group communication in distributed systems refers to communication between multiple nodes that are organized as a group.
2. In group communication, messages are sent to a group instead of sending separate messages to each individual node.
3. Group communication helps multiple computers coordinate their actions and share information efficiently.
4. Group communication is important for synchronization, collaboration, and maintaining data consistency.
5. Group communication is widely used in distributed databases, collaborative applications, and cloud systems.
6. Group communication improves reliability because messages can be delivered to multiple nodes.
7. Group communication also supports scalability because it allows communication with many nodes at once.
8. An example of group communication is sending updates to multiple servers in a distributed system.

Importance of Group Communication

1. Multiple nodes in distributed systems must work together to perform tasks.
2. Group communication allows quick sharing of information among all nodes.
3. Group communication helps maintain consistency of data across different machines.
4. Group communication improves reliability by replicating messages to multiple nodes.
5. Group communication supports system growth without reducing performance.
6. An example is real-time collaboration tools where updates are sent to all users simultaneously.

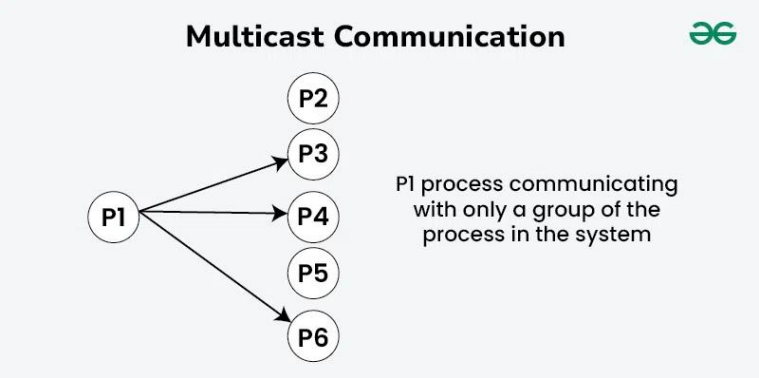
Types of Group Communication

1. Unicast Communication



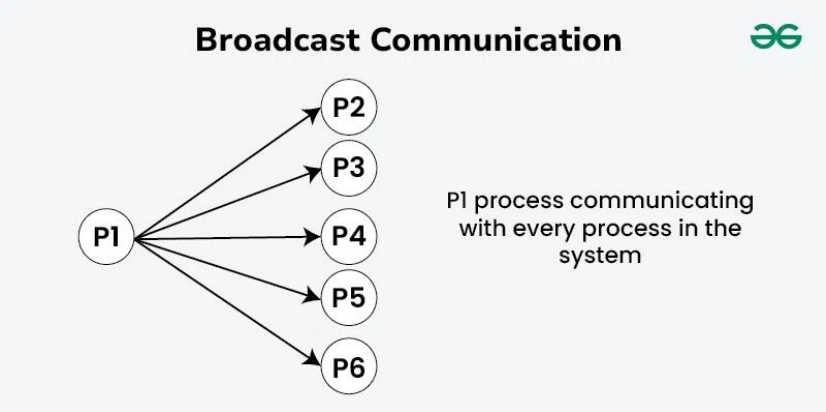
1. Unicast communication is point-to-point communication between one sender and one receiver.
2. The sender sends a message directly to a specific node using its address.
3. Unicast is simple and efficient for communication between two nodes.
4. Unicast is commonly used in client-server communication.
5. An example is a web browser sending a request to a web server.

2. Multicast Communication



1. Multicast communication is communication where one sender sends a message to multiple selected receivers.
2. The sender sends the message once, and it is delivered to all members of the group.
3. Multicast saves network bandwidth compared to sending separate messages.
4. Multicast is useful in distributed applications requiring updates to many nodes.
5. An example is live video streaming to multiple users.

3. Broadcast Communication



1. Broadcast communication is communication where one sender sends a message to all nodes in the network.
2. Every node in the system receives the message regardless of whether it needs it.
3. Broadcast is useful for network announcements or system updates.
4. Broadcast may cause network congestion in large networks.
5. An example is sending network configuration updates to all devices.

1-M (One-to-Many) Group Communication

1. One-to-Many communication means one sender sends messages to multiple receivers.
2. This type of communication is commonly implemented using multicast or broadcast methods.
3. The sender transmits a single message that is delivered to many nodes.
4. One-to-Many communication improves efficiency because repeated messages are avoided.
5. One-to-Many communication is useful in distributed notifications and updates.
6. An example is a server sending updates to multiple client computers.

M-1 (Many-to-One) Group Communication

1. Many-to-One communication means multiple senders send messages to a single receiver.
2. The receiver collects information from multiple nodes in the system.
3. This type of communication is useful for data aggregation and monitoring.
4. Many-to-One communication is commonly used in distributed data collection systems.
5. The receiver must handle multiple incoming messages efficiently.
6. An example is multiple sensors sending data to a central server.

Q. Explain Various Forms of Message Oriented Communication with Suitable Example

=>

Message Oriented Communication

1. Message oriented communication is a method of communication in distributed systems where processes exchange information in the form of messages.
2. A message is a block of data sent from one process to another through a network.
3. Message oriented communication does not require processes to share memory.
4. This communication method is widely used in distributed systems because machines are located at different places.
5. Message oriented communication provides flexibility, scalability, and reliability.
6. Examples include email systems, chat applications, and distributed databases.

Various Forms of Message Oriented Communication

1. Synchronous Communication

1. In synchronous communication, the sender waits for the receiver to receive the message and send a response.
2. The sender and receiver must be active at the same time for communication to occur.
3. Synchronous communication is similar to a telephone conversation.
4. This communication method ensures immediate response.
5. However, it may cause delays if the receiver is not available.
6. An example is a client sending a request to a server and waiting for a reply immediately.

2. Asynchronous Communication

1. In asynchronous communication, the sender sends the message and continues its work without waiting for a response.
2. The receiver processes the message whenever it becomes available.
3. The sender and receiver do not need to be active at the same time.
4. Asynchronous communication improves system performance and flexibility.
5. Message queues are commonly used for asynchronous communication.
6. An example is email communication where the sender does not wait for the receiver to read the message.

3. Persistent Communication

1. In persistent communication, messages are stored in the communication system until they are delivered successfully.
2. The system ensures message delivery even if the receiver is temporarily unavailable.
3. Persistent communication improves reliability in distributed systems.

4. Messages remain stored until the receiver retrieves them.
5. An example is message queue systems like RabbitMQ storing messages until processed.

4. Transient Communication

1. In transient communication, messages are delivered only if the receiver is active at the time of sending.
2. If the receiver is not available, the message is lost.
3. Transient communication is faster but less reliable compared to persistent communication.
4. This method is suitable for real-time systems.
5. An example is live video streaming where lost packets are not retransmitted.

5. Direct Communication

1. In direct communication, the sender sends messages directly to a specific receiver.
2. The sender must know the address or identity of the receiver.
3. Direct communication is simple and efficient for point-to-point communication.
4. This method is commonly used in client-server systems.
5. An example is a browser sending a request directly to a web server.

6. Indirect Communication

1. In indirect communication, messages are sent through an intermediary such as a message queue or mailbox.
2. The sender does not need to know the identity of the receiver.
3. Indirect communication provides flexibility and decouples sender and receiver.
4. Multiple receivers may access the same message source.
5. An example is a task queue where workers pick jobs from a queue.

7. Blocking Communication

1. In blocking communication, the sender waits until the message is received or processed.
2. Blocking communication ensures synchronization between processes.
3. It may reduce system performance due to waiting time.
4. Blocking communication is often used in synchronous systems.
5. An example is waiting for confirmation after sending a payment request.

8. Non-Blocking Communication

1. In non-blocking communication, the sender continues execution without waiting for the receiver.
2. Non-blocking communication improves concurrency and system efficiency.
3. The sender may check later whether the message was received.
4. This method is commonly used in asynchronous systems.
5. An example is sending notifications without waiting for acknowledgment.